FUNCTIONS

Functions are one of the most important tools in programming. They allow us to divide a program into smaller, logically connected units with a clearly defined purpose. A function is essentially a block of code that performs a specific task and can be called multiple times from different parts of the program. Instead of repeating the same code over and over, we can write it once inside a function and use it whenever needed.

In this way, the program becomes clearer, shorter, and easier to maintain. Imagine a program that needs to calculate the area of a circle in ten different places. Without functions, we would have to write the formula for the area ten times, whereas with a function we write the formula only once and simply call it whenever needed.

Functions enable **code reusability**, which is a key feature of good programming. They also contribute to clean and organized code. Instead of one large "monolithic" program, we have a collection of smaller, clearly defined components, each performing its own role. This makes the program easier to understand, test, and debug.

Functions in Python

In Python, functions are defined using the def keyword, followed by the function name and a list of parameters in parentheses. This is followed by a colon, and the function body is written indented so that Python knows which part of the program belongs to that specific function.

Example 4.1:

```
def hello():
        print("Hello World!")
```

A function is called simply by specifying its name followed by parentheses (including arguments if the function accepts any): hello().

A major advantage of functions is that we can pass data to them. This data is received by the function through **arguments**, while the function accesses them via **parameters**.

Parameters are the names listed in the function definition, acting like formal "entry points." Arguments are the actual values we provide when calling the function.

Example 4.2:

```
def square(number):
        return number*number
print(square(7)) #49
```

The value 7 is the **argument**. The function now works with this data and returns the result 49. Thus, a **parameter** is just a symbolic name that the function uses, while an **argument** is the actual value that "binds" to that name during execution.

As mentioned earlier, Python is a dynamically typed language, which means it determines data types at runtime. While this is very convenient, it can become problematic in larger programs. It is often difficult to

know what types of data a function actually expects and what it returns.

To clarify this, **type hints** were introduced. These are special annotations added to function parameters and

return values, describing the type of data a function accepts and returns. They do not change how Python executes the code—programs will run even without them—but using them makes code cleaner, more professional, and easier to maintain.

Example 4.3:

```
def square(number: int) -> int:
        return number*number
print(square(7)) #49
```

PROBLEM: Write a program that asks the user to enter a number and then checks whether that number is **prime**. A prime number is a natural number greater than 1 that is divisible only by 1 and itself.

SOLUTION:

```
def isPrime(number: int) -> bool:
if number < 2:
        return False
else:
        for i in range (2, number):
                if number % i == 0:
                        return False
return True

x: int = int(input("Enter a number: "))
if isPrime(x):
        print("Number is prime.")
else:
        print("Number is not prime.")
```

Lambda Functions

In Python, **lambda functions** are small, anonymous functions defined in a single line. They are especially useful when we want to create a simple function without using the standard def definition. Lambda functions can take multiple arguments but can contain only a single expression, the result of which is automatically returned. The name "lambda" comes from **lambda calculus**, a branch of mathematics that studies functions and their applications. In Python, lambda functions are used when we want to quickly write a function for one-time use or when we want to pass a function as an argument to another function.

The syntax of a lambda function is straightforward:

```
lambda argument1, argument2, ... : statement
```

Example 4.4:

```
def fun(f, number):
        return f(number)
result = fun(lambda x: x*3, 7)
print(result) #21
```

This program defines a function fun that takes two arguments: a function f and an integer number. When fun is called, it executes the function f on the integer number and returns the result. In this example, a lambda function lambda x: x*3 is passed as f, which takes a number and multiplies it by 3. The fun function calls this lambda function with the argument 7, resulting in 21, which is then printed to the screen. In short, the program simply uses a lambda function for a quick multiplication operation within the fun function.

Higher-Order Functions

Higher-order functions are functions that can take other functions as arguments or return functions as results. This feature allows for very powerful and flexible data manipulation, as we can create generic functions that work with different logic passed to them. In Python, common higher-order functions include map, filter, and reduce, which simplify the processing of collection structures such as lists or arrays.

The map function is a higher-order function that allows us to apply a given function to each element of a collection (e.g., a list) and obtain a new object with the transformed values. The result of map is a **map object**, which can be converted into a list, tuple, or another type of collection as needed.

Example 4.5:

```
numbers = [1, 2, 3, 4, 5]
squared = map(lambda x: x**2, numbers) # use lambda function on
every element
squared_list = list(squared) # convert map object into list
# squared_list -> [1, 4, 9, 16, 25]
```

The filter function is a higher-order function that allows selecting elements from a collection that meet a specific condition. The function we pass must return True or False. filter returns a **filter object**, which can be converted into a list, tuple, or another type of collection as needed.

Example 4.6:

```
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = filter(lambda x: x % 2 == 0, numbers) # keep only
even numbers
even_list = list(even_numbers) # convert filter object into list
# even_list -> [2, 4, 6]
```

The reduce function belongs to the functools module and is used to combine elements of a collection into a single value using a function that takes two arguments. Unlike map and filter, reduce does not return a collection; instead, it produces a single result after the function is applied recursively across all elements.

Example 4.7:

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
total = reduce(lambda x, y: x + y, numbers)
# total -> 15
```

A module in Python is a file containing Python code (typically functions, classes, and variables) that can be reused in other programs using the import statement.